

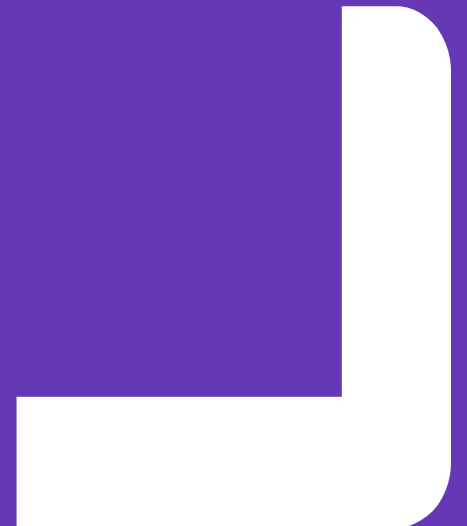
SOFTWARE ENGINEERING

Prof. Dr. Christoph Andriessens
christoph.andriessens@hs-weingarten.de





BUILD MANAGEMENT UND DEPENDENCY MANAGEMENT AM BEISPIEL VON MAVEN



Themen in dieser Vorlesung



"Bauen" von Software

- Software besteht aus mehr als ein paar Klassen:
 - eigene Bibliotheken und Teilprojekte
 - eingekaufte / fremde Bibliotheken
 - Abhängigkeiten zwischen den Teilprojekten
- Wie strukturiert man das Projekt?
- Wann muss kompiliert werden?
- Wann muss gepackt werden (jar, war, ear)?
- Wann muss wohin kopiert werden?

Abhängigkeiten

- Betriebssysteme
- Softwarebibliotheken
- Tools
- Datenbanken
- Laufzeitumgebungen
- Standorte (Internationalisierung)
- Bildschirmgrößen
- ...

Schritte der System-Erzeugung

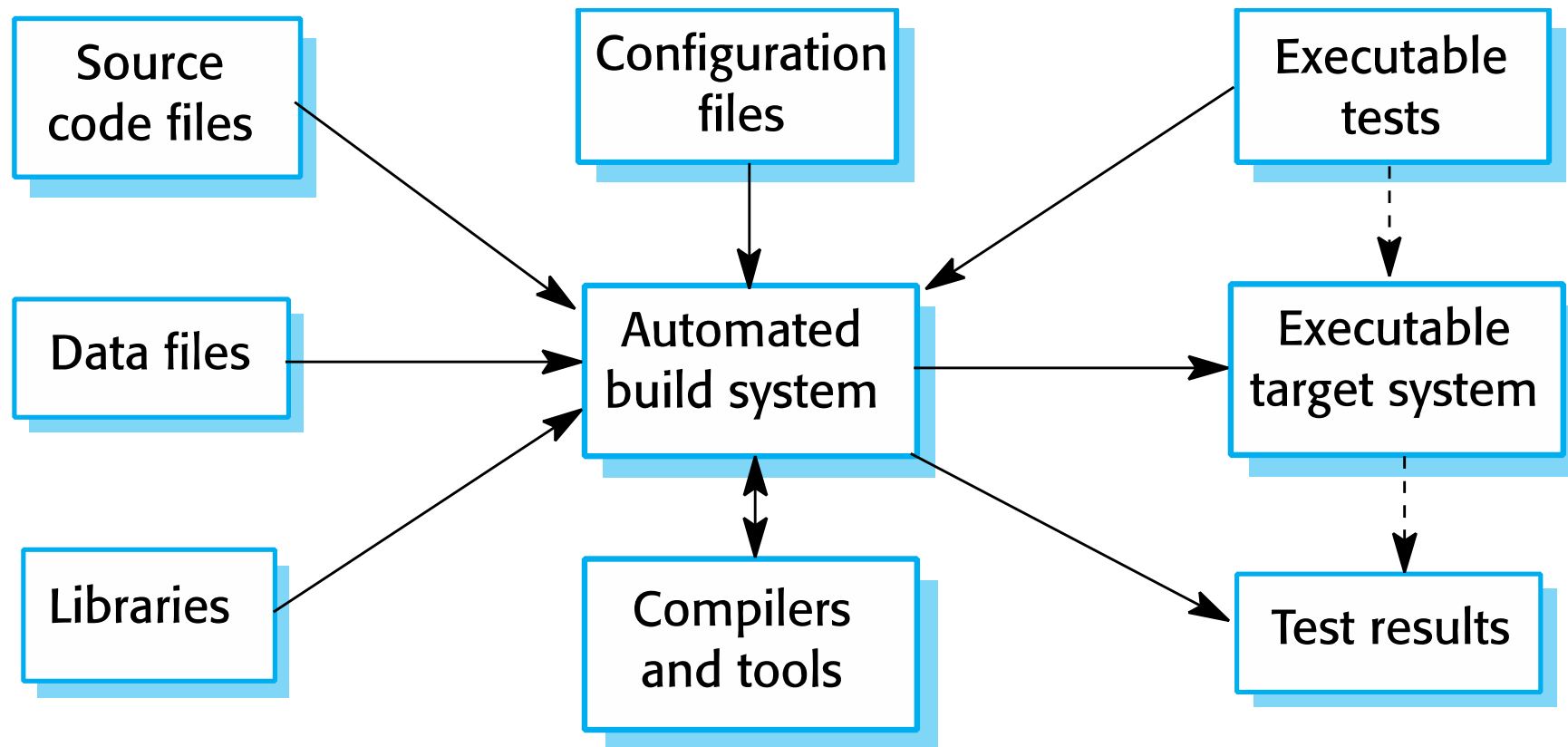
- Erzeugung kann viele Schritte mit bestimmten Reihenfolgen enthalten
 - Erst Teil A übersetzen, dann Skript B ausführen, dann Teil C unter Verwendung von A übersetzen, dabei unbedingt Version 1.10.9 (nicht 1.10.10) von Bibliothek D verwenden...
- Erzeugung kann lange dauern sein (=> läßt sie sich automatisieren und unbeobachtet ausführen?)
- Erzeugung kann beinhalten
 - Prüfschritte (Tests, Überwachung von Standards),
 - Berichterstellung
 - Erstellung von Dokumentation beinhalten

Build-Prozess

Im Build-Prozess wird ein vollständiges, ausführbares System erzeugt durch Übersetzung und Zusammenstellung ("linking") von Systemkomponenten, externen Bibliotheken, Konfigurationsdateien etc.

- Build-(Management-)Werkzeuge und Versionsverwaltungssysteme müssen miteinander kommunizieren: Im Build-Prozess werden Komponentenversionen aus Versions-Repository entnommen
- Das Build-Werkzeug verwendet außerdem eine Konfigurationsbeschreibung für die Erstellung des Systems

Build-Prozess und seine Bestandteile



Funktionalität eines Build-Systems

- Erzeugung (und Ausführung) eines Scripts für den Build des Systems
- Integration des Versionsverwaltungssystems
- Erzeugung eines ausführbaren Systems
 - Minimierung der Neuübersetzung von Komponenten
- Automatisierung von Tests
- Erzeugung der Dokumentation
- Erzeugung von Berichten über den Prozess

- Maven ist ein Build-Werkzeug – und organisiert entsprechend
 - die Verwendung benutzter Bibliotheken, Teilprojekte oder Komponenten,
 - das Bauen (Übersetzen, Linken) der Software,
 - das automatisierte Überprüfen und Testen der Software,
 - die Erzeugung von Dokumentationen (z.B. javadoc) und Messwerten oder Berichten,
 - die Zusammenstellung (z.B. jar-Erzeugung) für Auslieferung
 - die Organisation von Releases und deren Distribution mit standardisierter Wiederverwendung
- Deklarativer Ansatz: Festlegung, WAS passieren, nicht aber WIE es gemacht werden soll
- Einbindung von Plugins und Werkzeugen
- Website: <http://maven.apache.org>

Die Herkunft von Maven

- Maven (Jiddisch): „Wissensspeicher“
 - vgl. auch: Maven (engl., ugs.): „Experte“
- Ursprung: Vereinfachung der Build-Prozesse im Apache Jakarta Turbine-Projekt
 - Zahlreiche Teilprojekte
 - Jedes hat eigenes Ant-Build-Konfiguration, alle leicht unterschiedlich
 - JARs wurden in Versionsverwaltungssystem eingchecked
- *„We wanted a **standard way to build** the projects, a **clear definition of what the project consisted of**, an **easy way to publish project information** and a **way to share JARs across several projects**.“*

<http://maven.apache.org/what-is-maven.html>

Ziele von Maven

- **Grundeinsicht:** Alle Projekte teilen eine Reihe von Charakteristiken $\Rightarrow$ Ausnutzen!
 - Allgemein: Alle Projekte müssen gebaut, getestet, für die Verteilung eingepackt, dokumentiert und verteilt werden.
 - Im Detail: Eventuell viele Unterschiede – aber diese lassen sich entlang der Grenzen eines klar definierten Pfades ausdrücken!
- **Ziel:** Vereinfachung und Vereinheitlichung des „Build“-Prozesses durch Einsatz von Mustern bewährter Vorgehensweisen
- **Idee:** Hat man ein Maven-Projekt verstanden, versteht man alle und ist schnell im nächsten Projekt eingearbeitet

Einheitliche Maven-Vorlagen für Projekte

Beispiel einer Ordnerstruktur (1/2)

<code>src/main/java</code>	Application/Library sources
<code>src/main/resources</code>	Application/Library resources
<code>src/main/filters</code>	Resource filter files
<code>src/main/assembly</code>	Assembly descriptors
<code>src/main/config</code>	Configuration files
<code>src/main/scripts</code>	Application/Library scripts
<code>src/main/webapp</code>	Web application sources
<code>src/test/java</code>	Test sources
<code>src/test/resources</code>	Test resources
<code>src/test/filters</code>	Test resource filter files
<code>src/site</code>	Site
<code>LICENSE.txt</code>	Project's license
<code>NOTICE.txt</code>	Notices and attributions required by libraries that the project depends on
<code>README.txt</code>	Project's readme

Einheitliche Maven-Vorlagen für Projekte

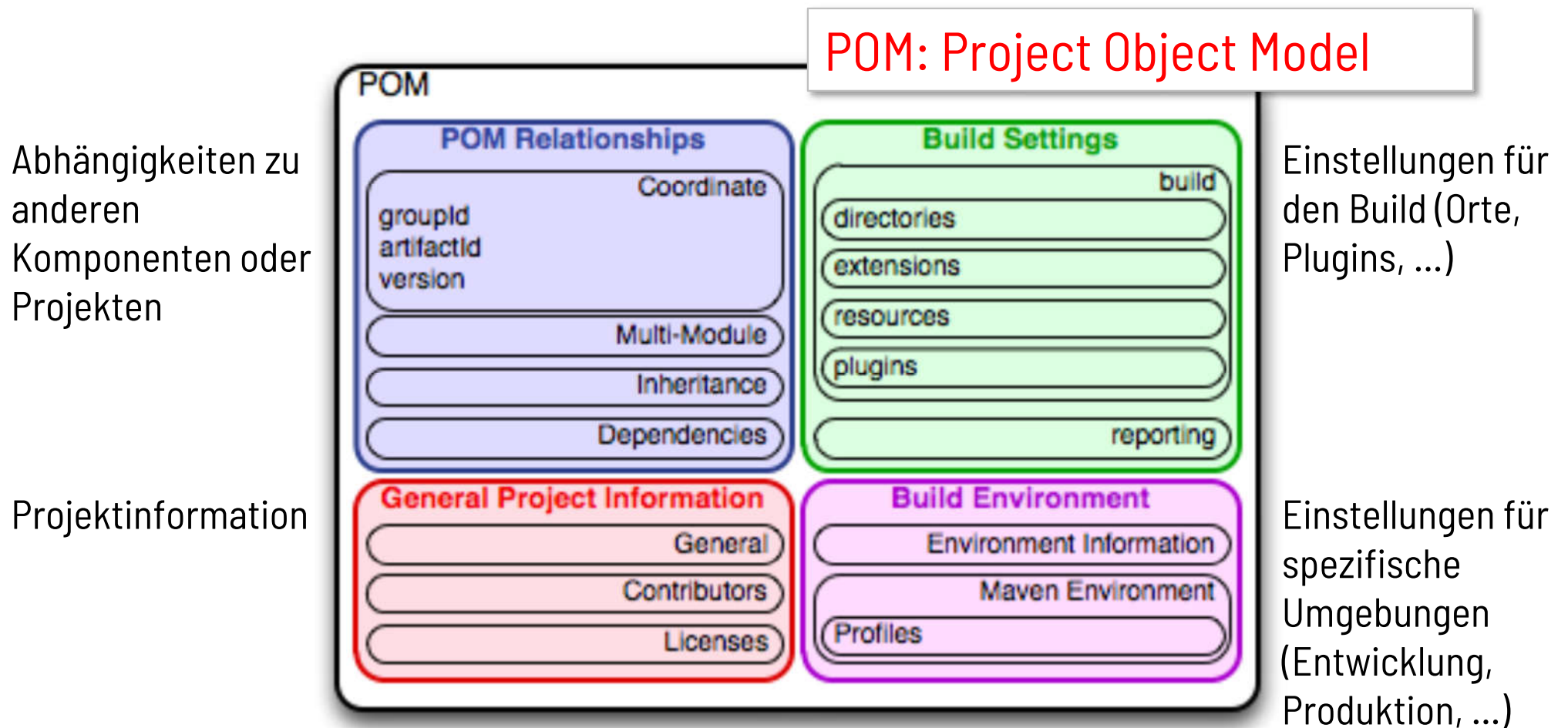
Beispiel einer Ordnerstruktur (2/2)

```
my-app
|-- pom.xml
'-- src
    |-- main
    |   |-- java
    |   |   |-- com
    |   |   |   |-- mycompany
    |   |   |   |   |-- app
    |   |   |   |   |   |-- App.java
    |-- test
    |   |-- java
    |   |   |-- com
    |   |   |   |-- mycompany
    |   |   |   |   |-- app
    |   |   |   |   |   |-- AppTest.java
```

← Quelltext der Anwendung

Deklarative Konfiguration eines Projekts

Datei pom.xml



Deklarative Konfiguration eines Projekts

Datei pom.xml: Einfaches Beispiel

```
1 <project xmlns="http://maven.apache.org/POM/4.0.0" xmlns:xsi="http://www.w3.org
2   xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 http://maven.apache.org
3   <modelVersion>4.0.0</modelVersion>
4   <groupId>de.hrw.swen</groupId>
5   <artifactId>swen-first-maven-app</artifactId>
6   <packaging>jar</packaging>
7   <version>1.0-SNAPSHOT</version>
8   <name>swen-first-maven-app</name>
9   <url>http://maven.apache.org</url>
10 <dependencies>
11   <dependency>
12     <groupId>junit</groupId>
13     <artifactId>junit</artifactId>
14     <version>3.8.1</version>
15     <scope>test</scope>
16   </dependency>
17 </dependencies>
18 </project>
```


Deklarative Konfiguration eines Projekts

Datei pom.xml in Maven-Eclipse-Integration

The screenshot shows the 'Overview' window in the Maven Eclipse Integration. The window is divided into several sections:

- Artifact**:
 - Group Id:
 - Artifact Id*:
 - Version:
 - Packaging:
- Project**:
 - Name:
 - URL:
 - Description:
 - Inception:
- Parent**:
- Properties**:
- Modules**: (with 'Create...' and 'Delete' buttons)
- Organization**:
- SCM**:
- Issue Management**:
- Continuous Integration**:

At the bottom, a tabbed interface is visible, with the following tabs: Overview, Dependencies, Plugins, Reporting, Dependency Hierarchy, Effective POM, and pom.xml. The 'pom.xml' tab is currently selected and highlighted with a red dashed border.

Deklarative Konfiguration eines Projektes

Maven-Koordinaten im POM

Eindeutige Identifizierung eines Projekts durch „Koordinaten-Tupel“:

- `groupId`: Angabe von Unternehmen oder Organisation
- `artifactId`: Identifiziert ein Projekt innerhalb der Organisation
- `version`: Die Angabe des zugehörigen Projekt-Releases
- `packaging`: Form der Bereitstellung

```
1 <project xmlns="http://maven.apache.org/POM/4.0.0"
2   xsi:schemaLocation="http://maven.apache.org/POM
3   <modelVersion>4.0.0</modelVersion>
4   <groupId>de.hrw.swen</groupId>
5   <artifactId>swen-first-maven-app</artifactId>
6   <packaging>jar</packaging>
7   <version>1.0-SNAPSHOT</version>
8   <name>swen-first-maven-app</name>
9   <url>http://maven.apache.org</url>
10  <dependencies>
11    <dependency>
12      <groupId>junit</groupId>
13      <artifactId>junit</artifactId>
14      <version>3.8.1</version>
15      <scope>test</scope>
16    </dependency>
17  </dependencies>
18 </project>
19
```

Deklarative Konfiguration eines Projektes

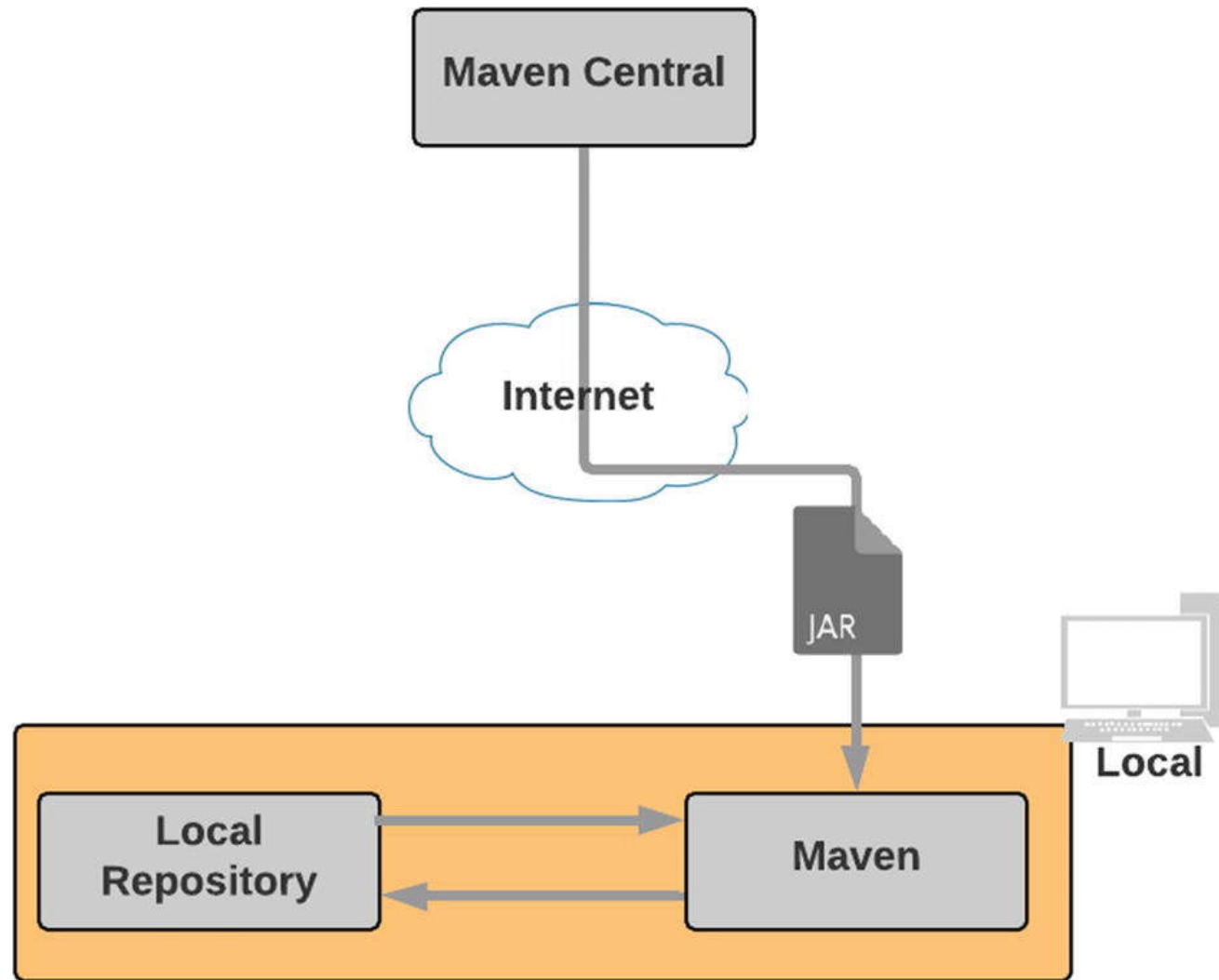
Abhängigkeiten im POM

- Alle Abhängigkeiten eines Projekts (verwendete Teilprojekte oder Komponenten) werden im POM angegeben
- Angabe durch die jeweiligen Maven-Koordinaten plus eines Gültigkeitsbereichs (scope)

```
1 <project xmlns="http://maven.apache.org/POM/4.0.0"
2   xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
3     http://maven.apache.org/maven-v4_0_0.xsd"
4   <groupId>de.hrw.swen</groupId>
5   <artifactId>swen-first-maven-app</artifactId>
6   <packaging>jar</packaging>
7   <version>1.0-SNAPSHOT</version>
8   <name>swen-first-maven-app</name>
9   <url>http://maven.apache.org</url>
10  <dependencies>
11    <dependency>
12      <groupId>junit</groupId>
13      <artifactId>junit</artifactId>
14      <version>3.8.1</version>
15      <scope>test</scope>
16    </dependency>
17  </dependencies>
18 </project>
```

Repositories-Architektur von Maven

Management von Abhängigkeiten

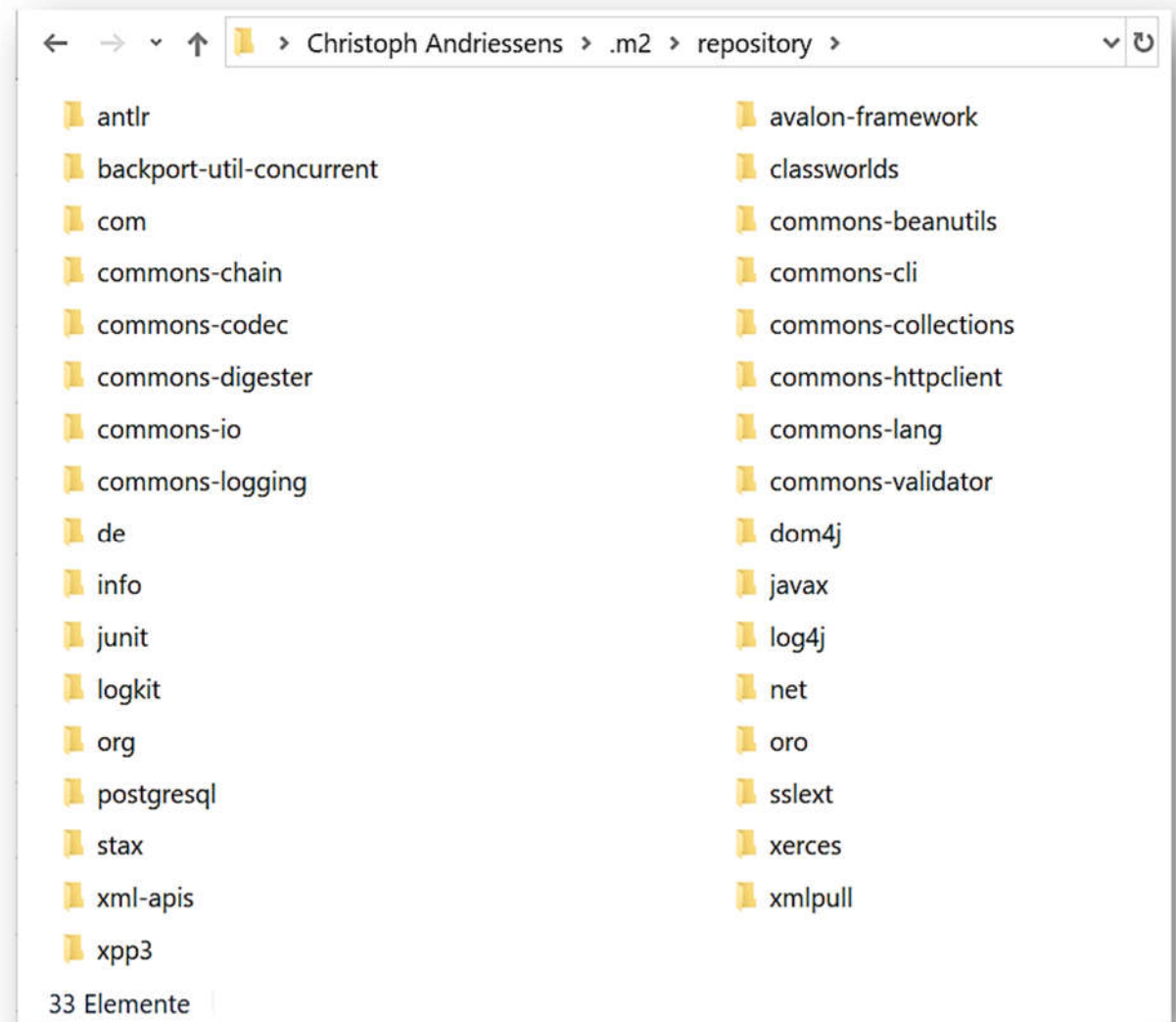


Varanasi, Balaji (2019): Introducing Maven. A Build Tool for Today's Java Developers. 2nd ed.: Apress.

Repository-Architektur von Maven

Struktur und lokales Repository

- Maven verwaltet alle erzeugten oder benutzten Artefakte (u.a. Bibliotheken, jars) in Repositorien
- Es gibt 2 Arten von Repositories:
 - Lokal (local)
 - Entfernt (remote)
- Ort des lokalen Repositoriums:
`%USERPROFILE%/.m2`



Repository-Architektur von Maven

Zentrales Repository „Central“

- Maven unterstützt in der Standard-Einstellung als zentrales Repository das entfernte Repository `Central` (<http://repo1.maven.org/maven2>)
- Dort sind etwa zahlreiche Open-Source-Bibliotheken als fertige Maven-Pakete enthalten und über ihre Maven-Koordinaten eingebunden werden
- Beispiel:

JSON Processing

JSR 353: Java API for JSON Processing - Reference Implementation

Dependencies

Umsetzungen von Java APIs wie `javax.json` sind als fertige Komponenten für Maven in `Central` hinterlegt.

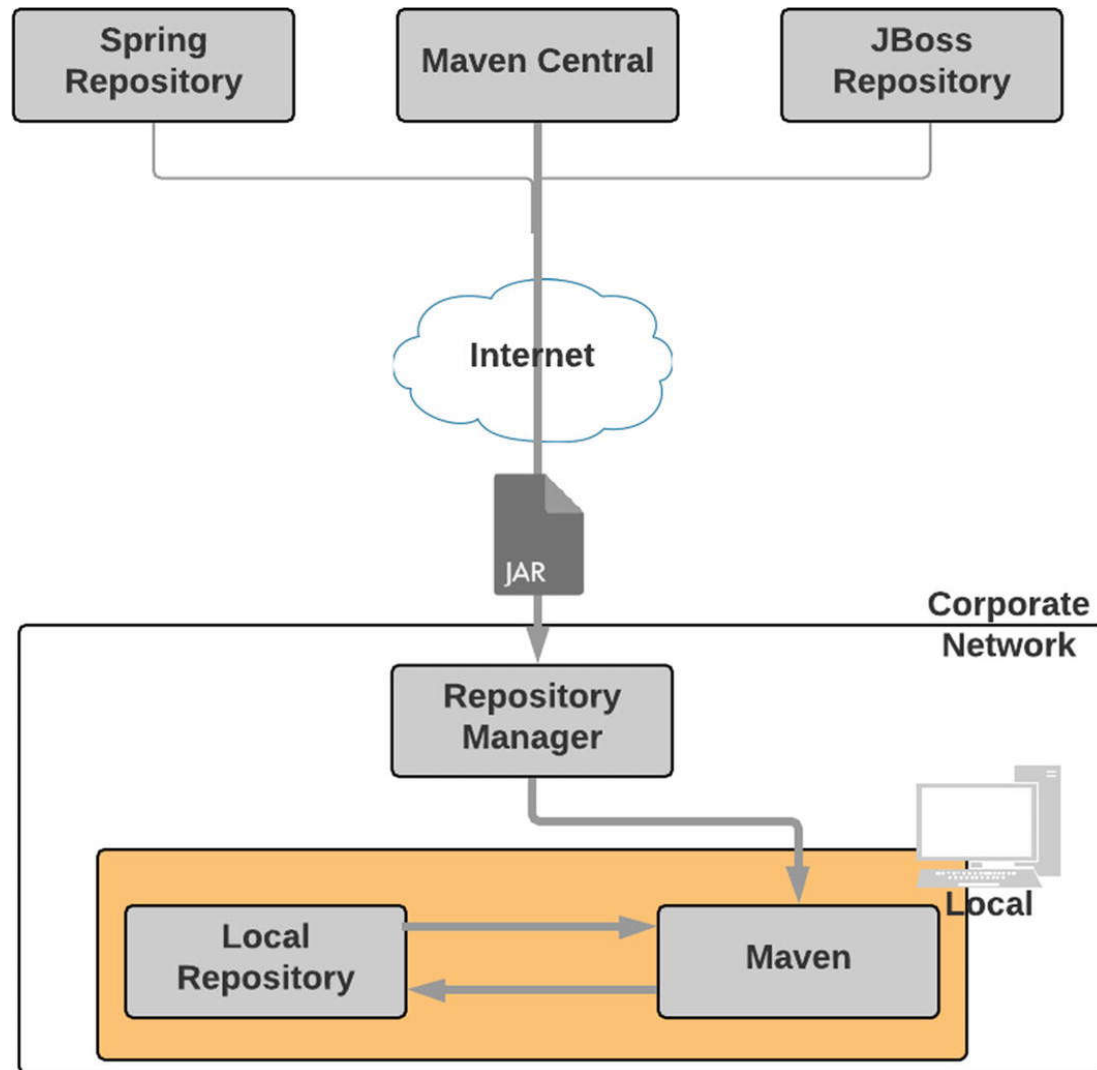
JSON Processing is built, assembled and installed using Maven. It is deployed to the Maven Central repository.

Developers using maven are likely to find it easier to include and manage dependencies of their applications. This document will explain to both maven and non-maven developers how to use JSON Processing in their applications.

In general, if you're not using Maven, you'd need to download dependencies (jar files) directly from the Maven repository and include it in the classpath.

Enterprise-Repositories-Architektur

Management von Artefakten und Abhängigkeiten



Varanasi, Balaji (2019): Introducing Maven. A Build Tool for Today's Java Developers. 2nd ed.: Apress.

Enterprise-Repositories-Architektur

Management von Artefakten und Abhängigkeiten

Unternehmen verwenden eigene lokale Repositories.

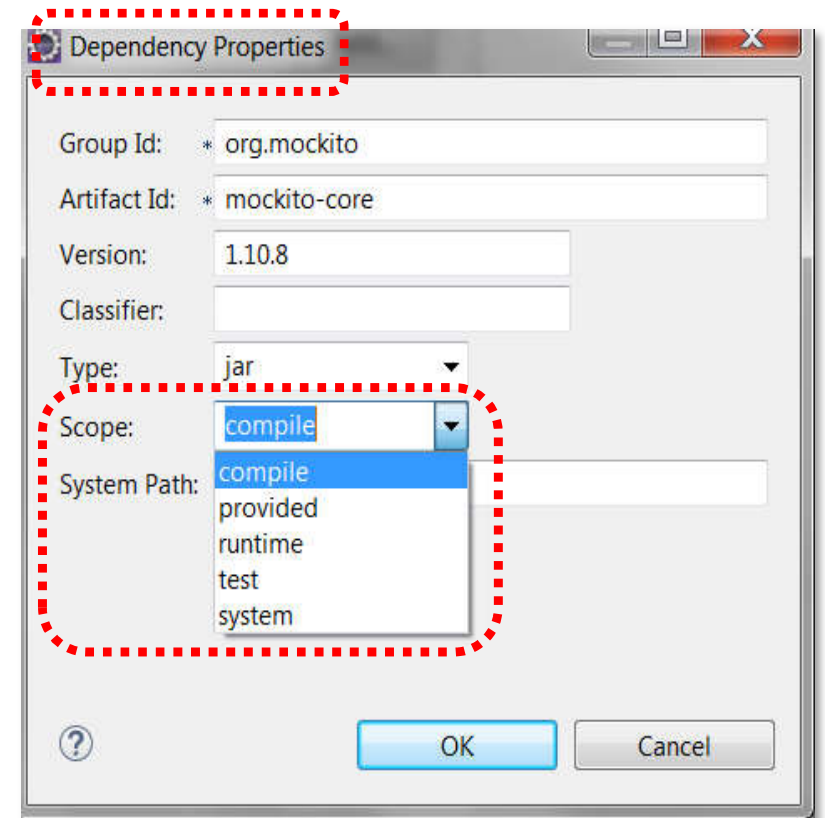
- Was wird sichtbar?
 - Welche Artefakte werden in welcher Version verwendet?
 - Abhängigkeiten von externen Artefakten zu eigenen Bibliotheken / Programmen.
- Vorgabe der zu verwendenden Repositories:
 - Kompatibilität
 - Bugfixes in externen Artefakten
 - Lizenzen
 - Coding Guidelines
 - Qualitätsstandards externer Artefakte

Deklarative Konfiguration eines Projektes

Gültigkeitsbereiche für Abhängigkeiten: Scopes

Der Scope steuert, welche Abhängigkeiten in welchem Classpath und in Anwendung enthalten sind:

- **compile**: Standard-Scope, immer verfügbar, wird paketiert.
- **provided**: Abhängigkeit, die bei Ausführung extern bereitgestellt wird (etwa JDK oder Java EE-Container). Verfügbar zur Übersetzungszeit, wird nicht paketiert.
- **runtime**: Benötigt für Ausführung und Test. Nicht verfügbar zur Übersetzungszeit, wird paketiert.
- **test**: Notwendig für Test, nicht normale Operation, deshalb verfügbar nur bei Test, Übersetzung und Ausführung. Wird nicht paketiert.
- **system**: Wie provided mit Artefakt aus lokalem Dateisystem, wird nicht in Repository nachgeschlagen. Verwendung nicht empfohlen.



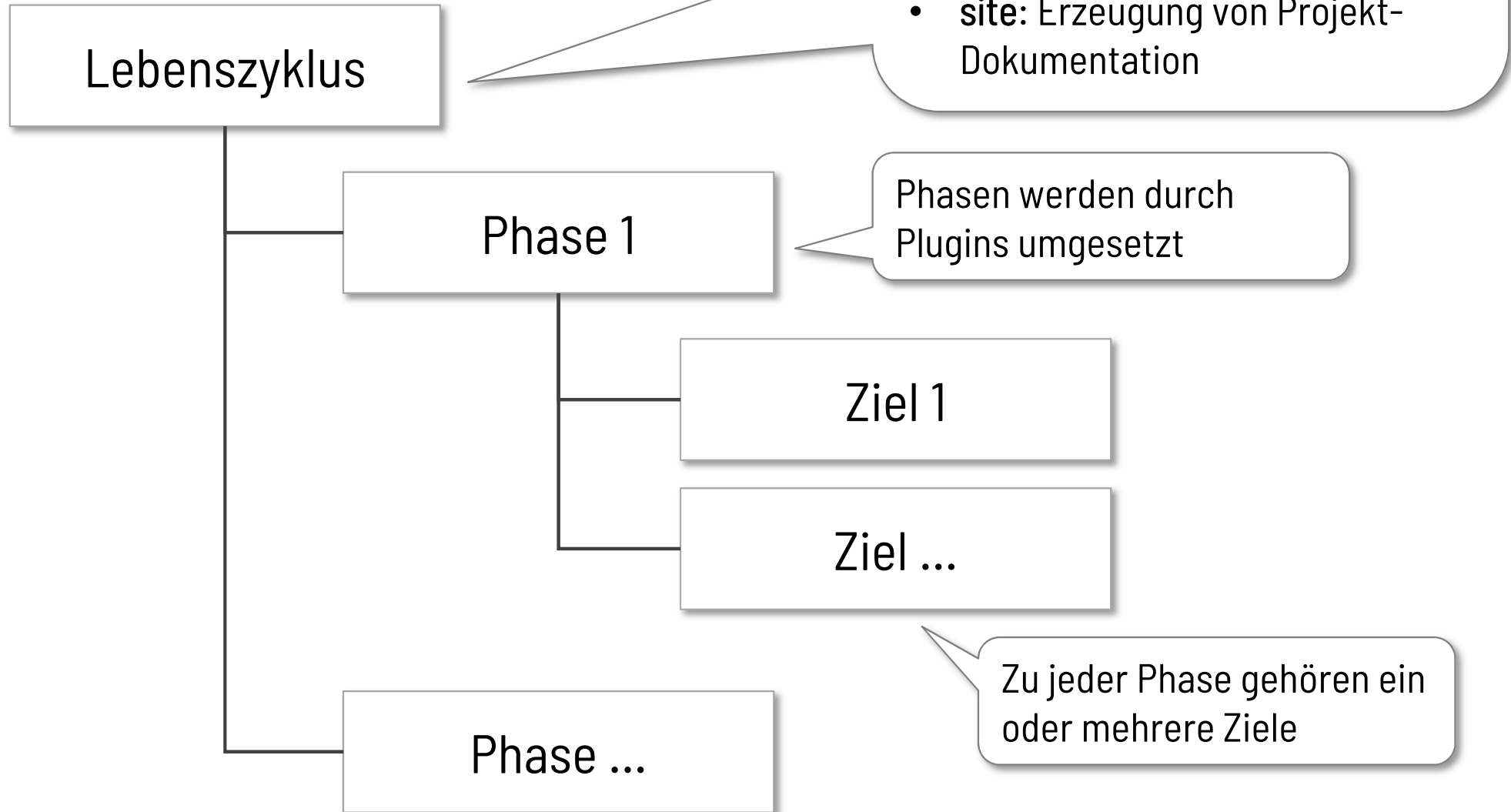
Deklarative Konfiguration eines Projektes

Versionsbezeichnungen bei Maven-Projekten

- **SNAPSHOT-Versionen** (etwa 0 . 0 . 1 –SNAPSHOT, allgemein jede Versionsnummer gefolgt vom Zusatzbezeichner **-SNAPSHOT**) sind
 - Builds eines Produkts in Entwicklung, nicht veröffentlicht und
 - können und dürfen sich laufend ändern, ohne dass Versionsnummer sich ändert
- **Versionen ohne Zusatzbezeichner** (Bsp: 1 . 2 . 3) gelten als veröffentlicht und dürfen sich **NIE** ändern
 - Verwendet ein Maven-Build Ergebnisse anderer Maven-Builds, dann:
 - Holt Maven SNAPSHOT-Versionen *immer* neu
 - Alle andere Versionen nur *einmal*

Der Build-Prozess von Maven

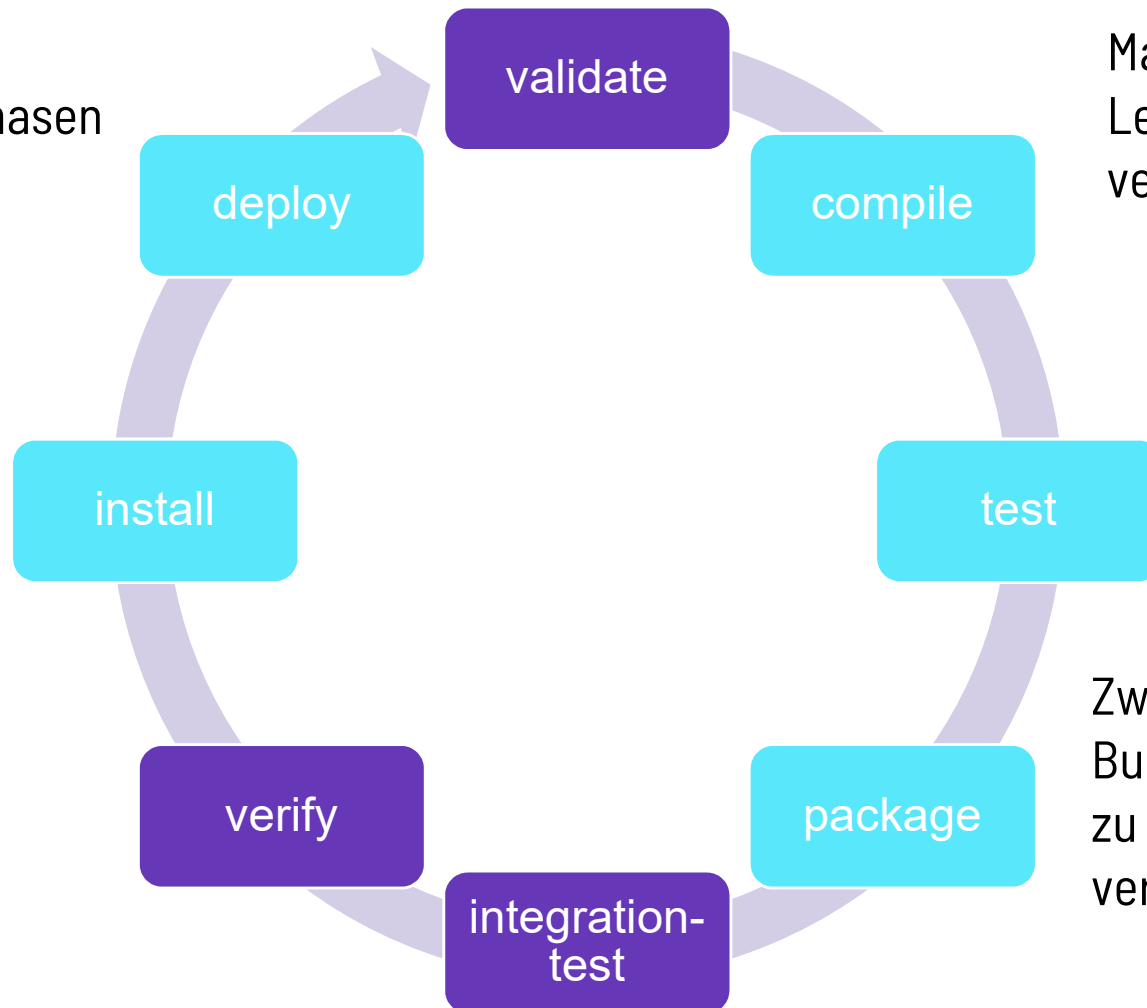
Lebenszyklen, Phasen, Ziele



Der Build-Prozess von Maven

Build-Lebenszyklus „default“ mit seinen Phasen

Hier: Nur eine **Auswahl** an Phasen von „default“!



Maven unterteilt den Lebenszyklus eines Builds in verschiedene **Phasen**.

Zweck der Phasen: Verschiedene Build-Operationen in **Reihenfolge** zu bringen und keine zu vergessen.

Der Build-Prozess von Maven

Wichtige Phasen des Build-Lebenszyklus

- **compile:** Quelltext des Projekts übersetzen
- **test:** Den übersetzten Quelltext mittels passendem Unit Testing-Framework testen
- **package:** Aus dem übersetzten Quelltext ein Paket zur Distribution erstellen, etwa ein JAR
- **install:** Das Paket in das lokale Repository installieren zur Verwendung in anderen lokale Projekten
- **deploy:** In Integrations- oder Releaseumgebung: Das finale Paket in ein entferntes Repository kopieren, um es mit anderen Entwicklern der Projekten zu teilen

Lebenszyklus clean hat Phase:

- **clean:** Artefakte früherer Builds des Projekts entfernen

Der Build-Prozess von Maven

„default“: Phasen mit Plugins und Zielen

Phase	Plugin	Goal
clean	Maven Clean plugin	clean
site	Maven Site plugin	site
process-resources	Maven Resources plugin	resource
compile	Maven Compiler plugin	compile
test	Maven Surefire plugin	test
package	Abhängig vom Packaging. Beispiel: Das Maven-JAR-Plugin	jar (bei Maven-JAR-plugin)
install	Maven Install plugin	install
deploy	Maven Deploy plugin	deploy

Eine einzelne Build-Operation bei Maven ist ein Goal (Ziel)

Bharathan, Raghuram (2015): Apache Maven Cookbook. Birmingham, UK: Packt Publishing.

Der Build-Prozess von Maven

Ausführung von Build-Operationen

Maven: Kommandozeilen-Werkzeug, 2 Aufrufvarianten:

- `mvn [plugin_identifizier] : [goal_identifizier]`

Führt nur *das eine* Ziel aus. Beispiele:

- `mvn compile:compile` - übersetzt das Projekt
- `mvn compile:testCompile` - übersetzt nur den Test-Code

- `mvn [phase]`

Führt *alle* Phasen bis zur angegeben aus. Beispiele:

- `mvn compile`: Übersetzt das Projekt
- `mvn package`: Wie `compile`, plus: führt Unit-Tests aus und erzeugt auslieferbares Artefakt
- `mvn verify`: Wie `package`, plus: Ausführung von Integrationstests und Qualitätsüberprüfungen für das Artefakt
- `mvn install`: Wie `verify`, plus: Ablage des Artefaktes in lokalem Repository

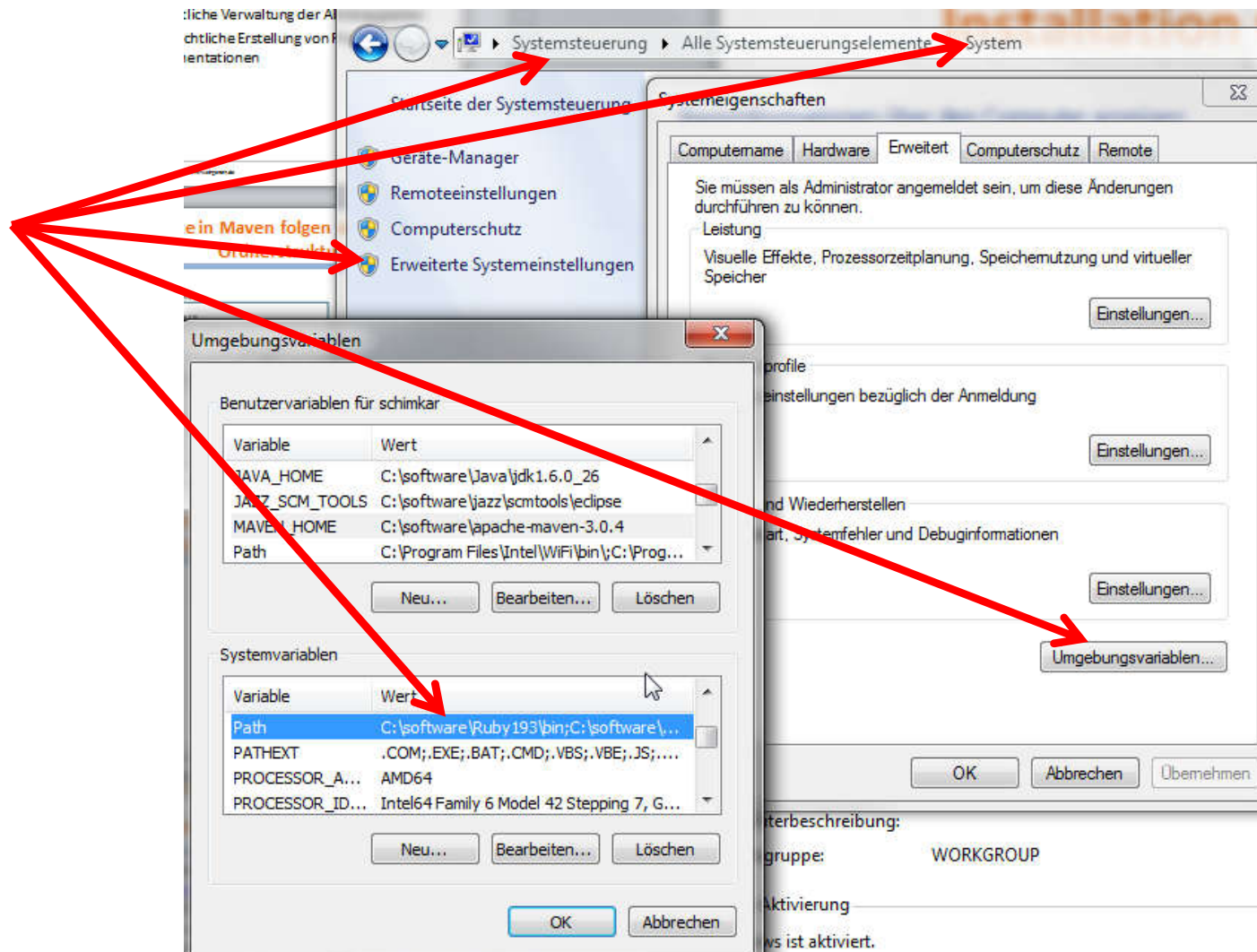
Exkurs: Installation von Maven

Voraussetzungen, Download, Konfiguration

- Java (JDK) sollte installiert sein
 - Aufruf von `java -version` zeigt die Version an
 - Download von Maven (als Binary-ZIP-Archiv)
 - <http://maven.apache.org/download.html>
 - Installation:
 - Maven lokal auf Festplatte entpacken
 - Unter Windows das Verzeichnis `Maven\bin` (mit kompletter Bezeichnung) in den PATH aufnehmen
- Siehe auch:
- <http://maven.apache.org/guides/getting-started/windows-prerequisites.html>

Exkurs: Installation von Maven

Konfiguration des PATH unter Windows



Exkurs: Einsatz von Maven

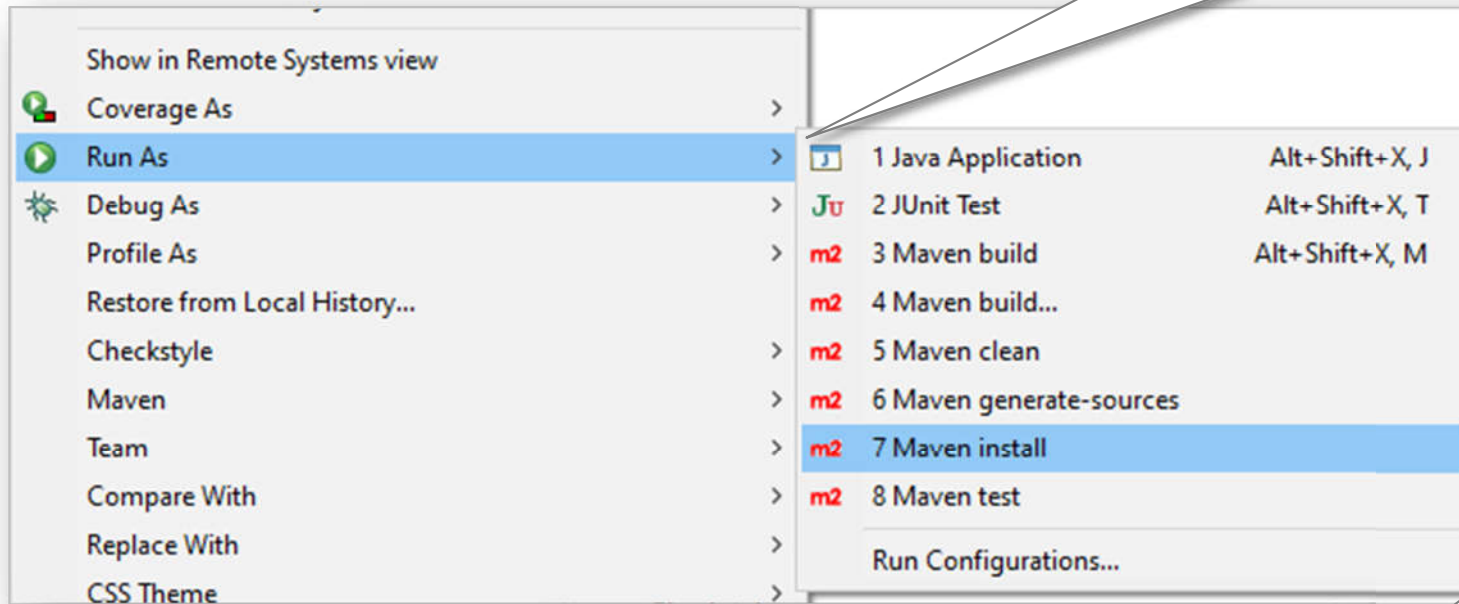
Verwendung mit Eclipse

- Eclipse-Plugin von Maven: Erzeugt Eclipse-IDE-Dateien aus einem POM. Beispiele:
 - `mvn eclipse:help` - zeigt alle Ziele an
 - `mvn eclipse:eclipse` - Erzeugt Konfigurationsdateien für Eclipse-Workspace
- Integration von Maven in Eclipse: M2Eclipse integriert Maven in Eclipse.
 - Website: <http://eclipse.org/m2e/>
 - Bereits integriert in Java EE-Edition von Eclipse

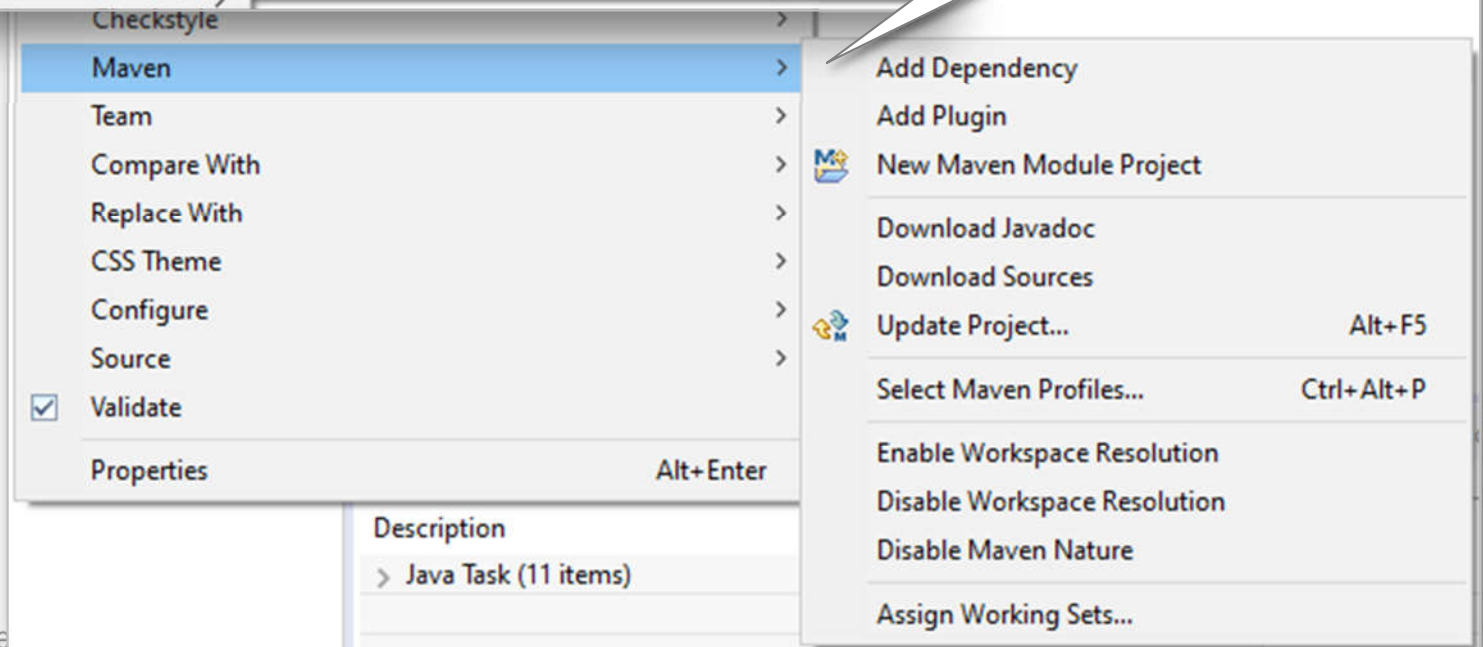
Exkurs: Einsatz von Maven

Verwendung in Eclipse

Aufruf von Maven-Phasen und -Zielen



Konfiguration von Maven und des Eclipse-Projekts





Maven: Beispiel

Einfaches Java-Projekt erzeugen

- Ein Java-Projekt mit Maven erzeugen
- Das Projekt übersetzen und ausführen
- Beispiel für ein Java-Projekt

```
mvn archetype:generate
  -DgroupId=de.hrw.swen
  -DartifactId=swen-first-maven-app
  -DarchetypeArtifactId=maven-archetype-quickstart
  -DarchetypeVersion=1.4
  -DinteractiveMode=false
```

- Beispiel für ein Web-Projekt

```
mvn archetype:generate
  -DgroupId=de.hrw.swen
  -DartifactId=swen-first-maven-app
  -DarchetypeArtifactId=maven-archetype-webapp
  -DinteractiveMode=false
```



Maven: Beispiel

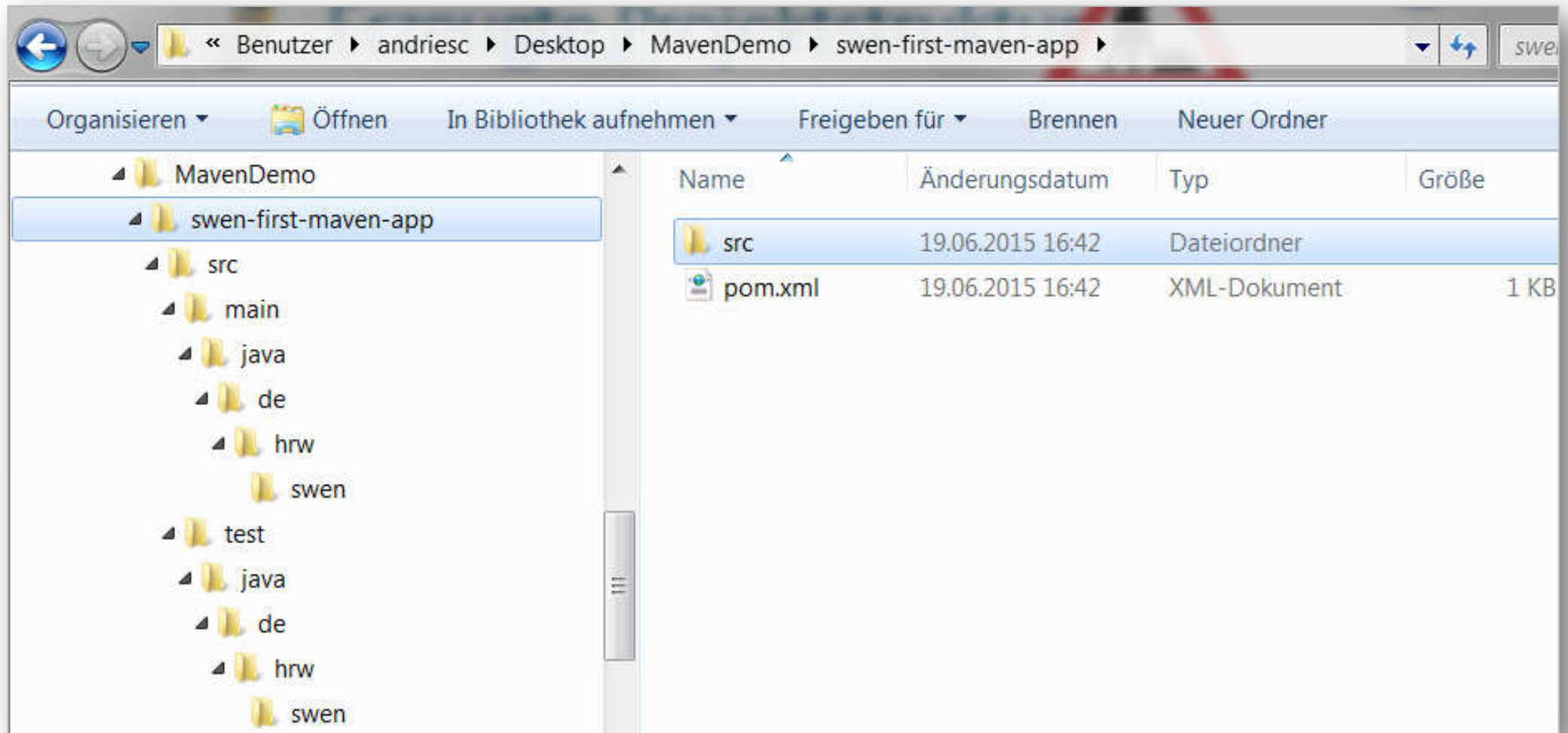
Maven bereitet alles vor

```
Downloaded: http://repo.maven.apache.org/maven2/org/codehaus/groovy/groovy/1.8.3/groovy-1.8.3.jar (5394 KB at 50.7 KB/sec)
[INFO] Generating project in Batch mode
Downloading: http://repo.maven.apache.org/maven2/org/apache/maven/archetypes/maven-archetype-quickstart/1.0/maven-archetype-quickstart-1.0.jar
Downloaded: http://repo.maven.apache.org/maven2/org/apache/maven/archetypes/maven-archetype-quickstart/1.0/maven-archetype-quickstart-1.0.jar (5 KB at 16.8 KB/sec)
Downloading: http://repo.maven.apache.org/maven2/org/apache/maven/archetypes/maven-archetype-quickstart/1.0/maven-archetype-quickstart-1.0.pom
Downloaded: http://repo.maven.apache.org/maven2/org/apache/maven/archetypes/maven-archetype-quickstart/1.0/maven-archetype-quickstart-1.0.pom (703 B at 2.9 KB/sec)
[INFO] -----
[INFO] Using following parameters for creating project from Old (1.x) Archetype: maven-archetype-quickstart:1.0
[INFO] -----
[INFO] Parameter: groupId, Value: de.hrw.swen
[INFO] Parameter: packageName, Value: de.hrw.swen
[INFO] Parameter: package, Value: de.hrw.swen
[INFO] Parameter: artifactId, Value: swen-first-maven-app
[INFO] Parameter: basedir, Value: C:\Users\schimkar\Documents\vorlesungen\__ss13\swen\beispiel
[INFO] Parameter: version, Value: 1.0-SNAPSHOT
[INFO] project created from Old (1.x) Archetype in dir: C:\Users\schimkar\Documents\vorlesungen\__ss13\swen\beispiel\swen-first-maven-app
[INFO] -----
[INFO] BUILD SUCCESS
[INFO] -----
[INFO] Total time: 22.979s
[INFO] Finished at: Mon May 13 10:38:33 CEST 2013
[INFO] Final Memory: 9M/123M
[INFO] -----
C:\Users\schimkar\Documents\vorlesungen\__ss13\swen\beispiel>
```




Maven: Beispiel

Erzeugte Projektstruktur





Maven: Beispiel

Erzeugtes Project Object Model – pom.xml

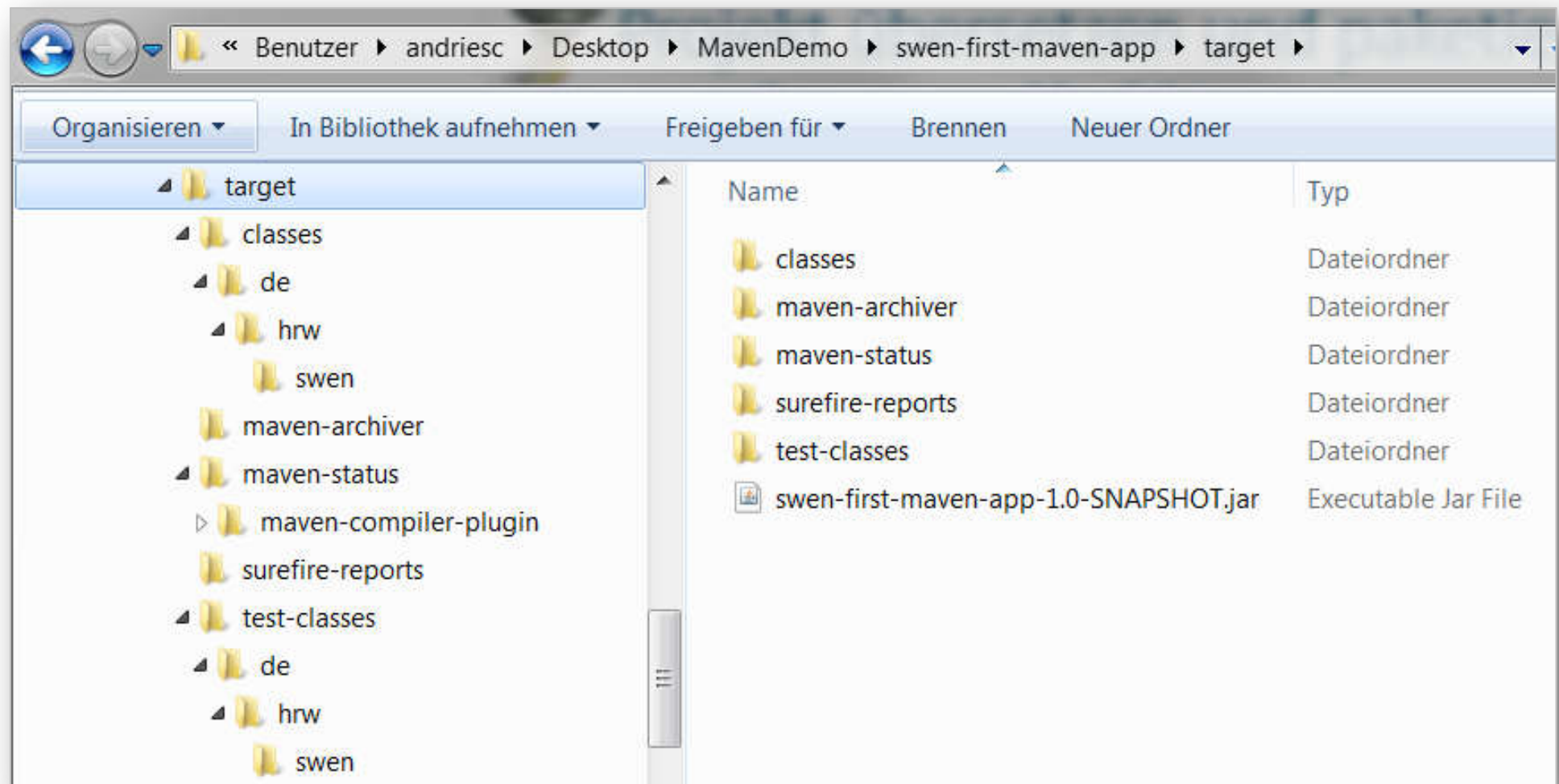
```
- <project xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 http://maven.apache.org/maven-v4_0_0.xsd">
  <modelVersion>4.0.0</modelVersion>
  <groupId>de.hrw.swen</groupId>
  <artifactId>swen-first-meven-app</artifactId>
  <packaging>jar</packaging>
  <version>1.0-SNAPSHOT</version>
  <name>swen-first-meven-app</name>
  <url>http://maven.apache.org</url>
- <dependencies>
  - <dependency>
    <groupId>junit</groupId>
    <artifactId>junit</artifactId>
    <version>3.8.1</version>
    <scope>test</scope>
  </dependency>
</dependencies>
</project>
```



Maven: Beispiel

Projekt übersetzen und paketieren

`mvn package`





Maven: Beispiel

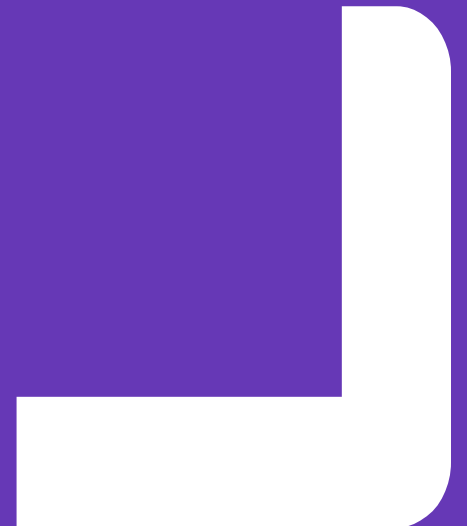
compile, test, paketieren und ausführen

- `mvn package`
- Aufruf über Java:
`java -cp target/swen-first-maven-app-1.0-SNAPSHOT.jar de.hrw.swen.App`

```
C:\Users\andriesc\Desktop\MavenDemo\swen-first-maven-app>java -cp target/swen-first-maven-app-1.0-SNAPSHOT.jar de.hrw.swen.App
Hello World!
C:\Users\andriesc\Desktop\MavenDemo\swen-first-maven-app>
```

- Aufruf mit Maven:
`mvn exec:java -Dexec.mainClass=de.hrw.swen.App`
- Jeweils Ausgabe auf Console: `Hello World!`

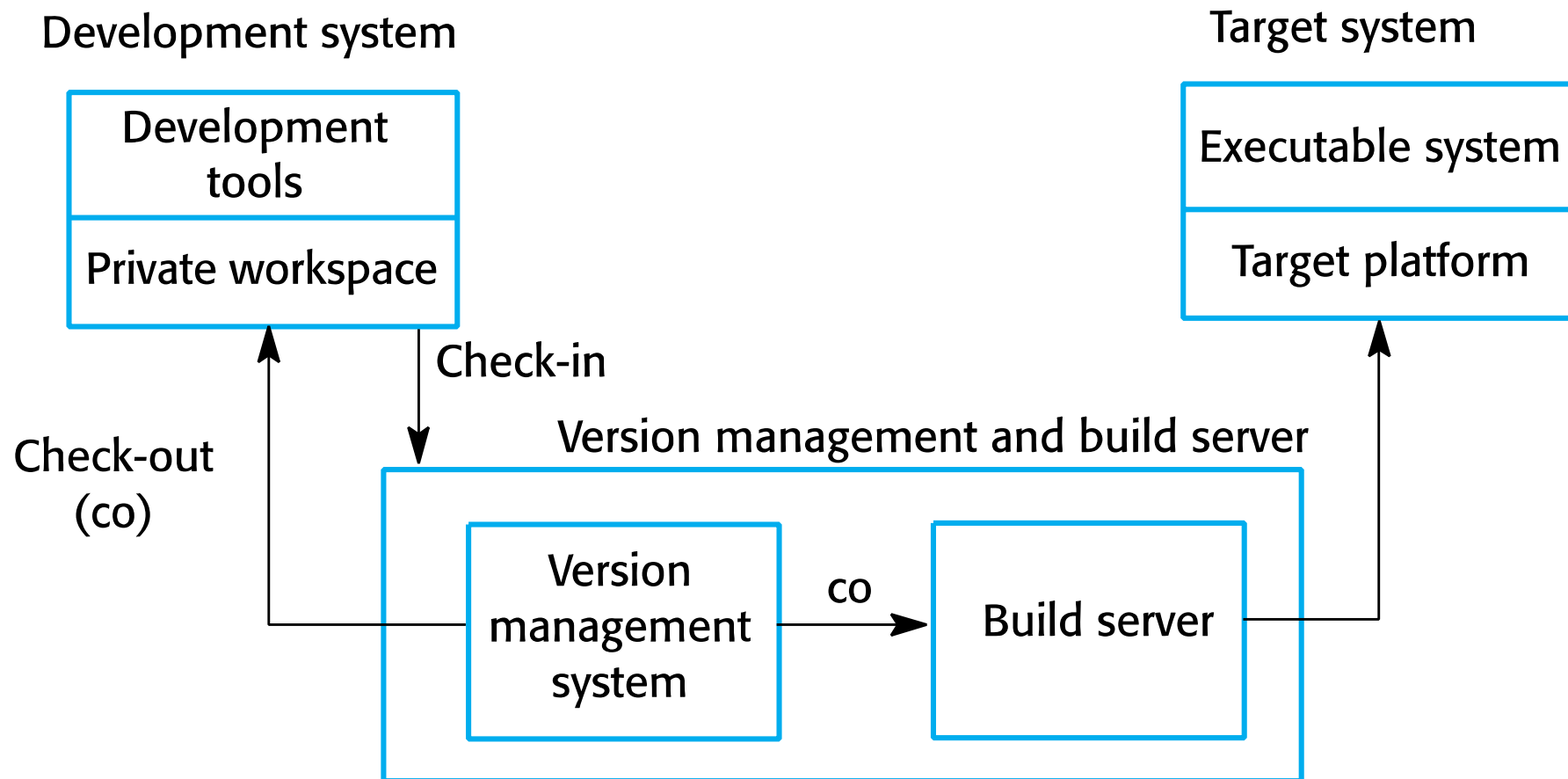
┌ BUILD-PROZESS ALLGEMEIN



Verschiedene System-Plattformen sind beteiligt

- Das **Entwicklungssystem** mit Entwicklungswerkzeugen wie Compiler (Übersetzer), Quelltexteditoren etc.
- Der **Build-Server**, der zum Bauen verwendet wird und eine endgültige, definitive Systemversion enthält
- Das **Zielsystem**, auf dem das System ausgeführt wird
 - Bei Echtzeitsystemen und Embedded Systems ist das Zielsystem meistens kleiner und einfacher als das Entwicklungssystem (etwa bei Mobiltelefonen)

Entwicklungs-, Build- und Zielplattformen



Build-Werkzeuge

- Urvater: **make** (Stuart Feldman, 1977)
 - Unix
 - Zahlreiche angelehnte / abgeleitete Werkzeuge
- Etwas Neuer: **Ant** (Apache Software Foundation, 2000)
 - Deklarative Anweisungen in XML-Datei
 - Jedes Teilprojekt bindet eigene Abhängigkeiten ein
- Aktuell: **Maven** (Apache Software Foundation, 2002)
 - Deklarativ
 - Hauptsächlich Java-Welt
- Aktueller: **Gradle** (Gradle Inc.), **Buildr** (Apache Software Foundation), ...

Minimierung von Neuübersetzungen

- Build-Managementwerkzeuge versuchen meistens, die Anzahl der benötigten Compile-Schritte zu minimieren
- Dazu wird geprüft, ob noch eine übersetzte Version der Komponente verfügbar ist: Falls ja, entfällt deren Übersetzung
 - Die Versionen von Quelltext und übersetztem Code („*object code*“) wird durch eine **eindeutige Signatur** gekennzeichnet, die sich bei Veränderung des Quelltexts ändert
 - Durch Vergleich der Signaturen von Quelltext und übersetztem Code kann die Entscheidung über Neuübersetzung getroffen werden

Minimierung von Neuübersetzungen

Eindeutige Signaturen für Dateien

- **Zeitstempel der letzten Modifikation**
 - Hier wird Zeit und Datum der letzten Dateiveränderung als Signatur verwendet. Wenn die Quelltextdatei einer Komponente nach der letzten Übersetzung verändert wurde, wird neu übersetzt.
- **Quelltext-Prüfsummen**
 - Als Signatur wird eine Prüfsumme verwendet, die über den Quelltext berechnet wird. Selbst bei Veränderung von nur einem Zeichen ändert sich diese Prüfsumme. Neue Prüfsumme führt zu neuer Übersetzung.

Prozessfragen

Versionsverwaltung organisiert Versionen und gleichzeitige Arbeit mehrerer Entwickler.

Build-Management automatisiert Erzeugung und Prüfung (inkl. Test) des Systems.

Zu welchen Zeitpunkten...

- ...übergeben Entwickler ihre (Zwischen-)Ergebnisse an die Versionsverwaltung?
- ...wird das System im Build-Management erzeugt?

Prozessfragen

Täglicher Build

Es wird eine Deadline festgesetzt: Etwa 14:00 Uhr

- Wenn Entwickler neue Komponentenversionen fertig haben, müssen sie diese bis zu dieser Uhrzeit abliefern
- Ab der Deadline (also hier 14:00 Uhr) wird dann eine neue Systemversion aus den Lieferungen gebaut
- Die neue Systemversion wird von einem Testteam mit vordefinierten Tests getestet
- Entdeckte Fehler werden dokumentiert und an die Entwickler zur Bearbeitung gegeben. Die Entwickler müssen die Fehler für die Folgeversion beheben

Prozessfragen

Agiler Build-Prozess (1/2)

- Hole den Hauptentwicklungszweig („*mainline*“) aus dem Versionsverwaltungssystem in den lokalen Arbeitsbereich des Entwicklers
- Bevor Änderungen durchgeführt werden: Baue das System und führe die automatisierten Tests durch, um zu sehen ob alles in Ordnung ist
 - Bei Fehlschlag: Der Build ist kaputt („*build is broken*“), letzten Bearbeiter informieren, er ist für Reparatur verantwortlich
- Die gewünschten Änderungen durchführen
- Im lokalen Arbeitsbereich das System wieder bauen und Tests durchführen. Bei Fehlern korrigieren und solange durchführen, bis die Tests durchlaufen.

Prozessfragen

Agiler Build-Prozess (2/2)

- Die Änderungen in das gemeinsame Versionsmanagement übertragen
- Auf dem zentralen Build-Server das System neu bauen und die Tests ablaufen lassen
 - Falls zwischenzeitliche Modifikationen von anderen zu fehlschlagenden Tests führen: Diese Modifikationen in eigene Arbeiten integrieren, bis Tests durchlaufen
- Wenn Tests durchlaufen, die eigenen Änderungen ggf. als neue Version kennzeichnen
 - Übernahme in entsprechenden Zweig, Tagging, ...

Zusammenfassung

- Werkzeuge unterstützen und automatisieren den teilweise sehr komplexen Build-Prozess.
- Maven unterstützt neben einer Automatisierung den Build-Vorgang und die Projektorganisation mit zahlreichen Mustern und Vorlagen. Eine besondere Stärke sind die Organisation von benötigten Bibliotheken (Abhängigkeiten).
- Unterschiedliche Lizenzen von Bibliotheken können ein rechtlich-organisatorisches Problem sein.

